

Refatoração Automática de Classes de Teste Usando Métricas de Similaridade e Algoritmos de Agrupamento

Seminário de Andamento de Doutorado

Aluno: Lucas Pereira da Silva

Orientador: Prof. Dr. Ronaldo dos Santos Mello

Coorientadora: Prof.^a Dr.^a Patricia Vilain

Banca

Prof.^a Dr.^a Carina Friedrich Dorneles

Prof. Dr. Raul Sidnei Wazlawick

Roteiro

1. Introdução
2. Problema de Pesquisa
3. Fundamentação Teórica
4. Estado da Arte
5. Proposta
6. Conclusão

Introdução

O que é teste de software?

Introdução

Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

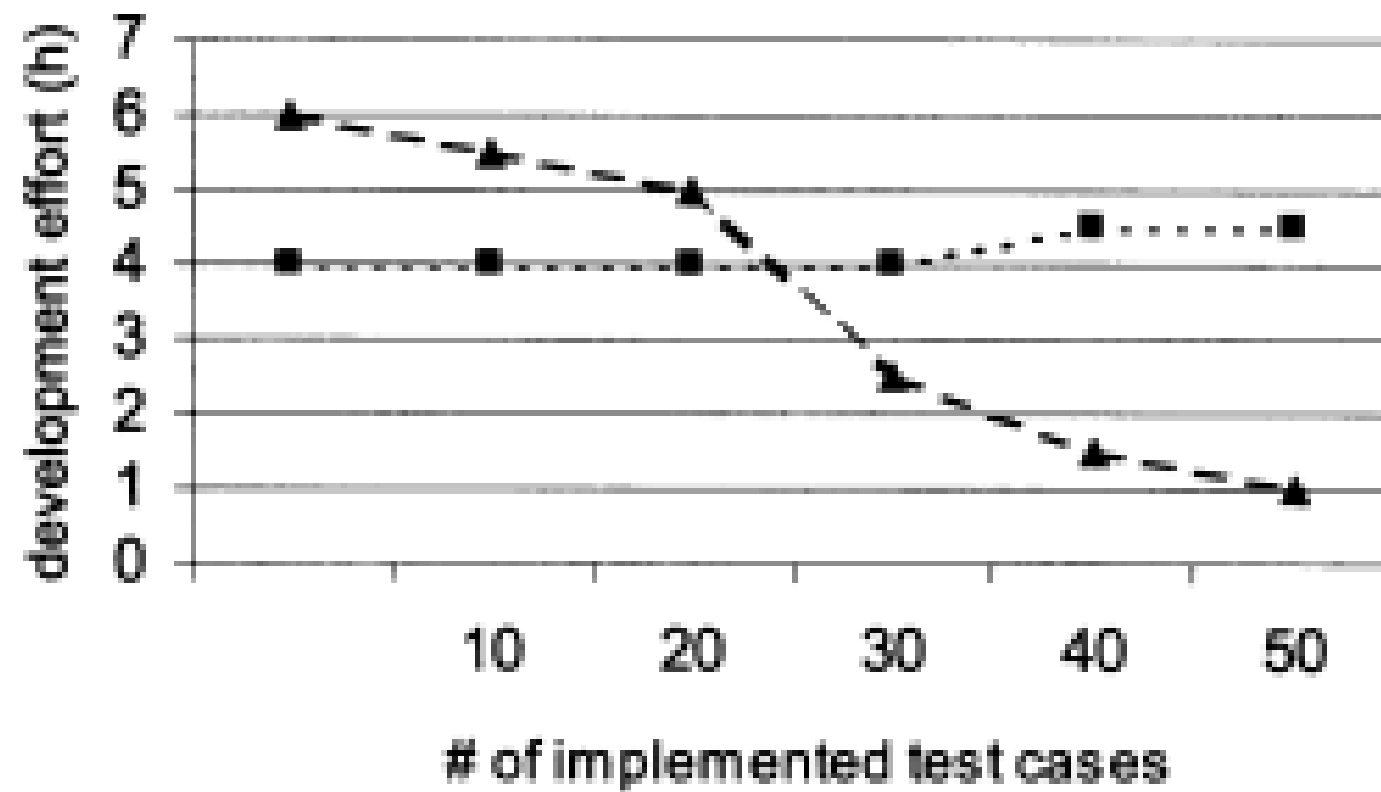
Proposta

Conclusão

A automatização de teste de software é o uso de ferramentas de software para controlar a execução de testes, comparar resultados esperados com reais e reportar falhas, reduzindo a intervenção humana em atividades repetitivas de teste (Sommerville, 2015).

O motivo comum pelo qual pessoas e organizações abandonam testes automatizados está no fato de que os testes se tornam frágeis e com alto custo de manutenção (Emery, 2009).

O sucesso a longo prazo da automatização de testes é fortemente influenciado pela manutenibilidade do código de teste (Greiler, van Deursen e Storey, 2013).



...■... development effort ad hoc approach
-▲- development effort with reusable components

Problema de Pesquisa

Testes tendem a ser repetitivos e apresentam um alto potencial de reuso. Conforme o software evolui e o número de testes aumenta é comum que a quantidade de código duplicado através dos testes também aumente (Berner, Weber e Keller, 2005).


```
1 class TesteSistemaBancarioComDuplicacao {
2     @Test
3     fun sacarValorDoSaldo() {
4         val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
5         val sistemaBancario = SistemaBancario()
6         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
7         val trindade = bb.criarAgencia("Trindade")
8         val maria = trindade.criarConta("Maria", "000.000.000-00")
9         val cincoReais = casaDaMoeda.emitir(5)
10        maria.depositar(cincoReais)
11        val saque = maria.sacar(cincoReais)
12        assertTrue(saque.sucesso())
13        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
14    }
15
16    @Test
17    fun sacarMaisQueOSaldo() {
18        val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
19        val sistemaBancario = SistemaBancario()
20        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
21        val trindade = bb.criarAgencia("Trindade")
22        val maria = trindade.criarConta("Maria", "000.000.000-00")
23        val cincoReais = casaDaMoeda.emitir(5)
24        maria.depositar(cincoReais)
25        val dezReais = casaDaMoeda.emitir(10)
26        val saque = maria.sacar(dezReais)
27        assertFalse(saque.sucesso())
28        assertEquals(cincoReais, maria.calcularSaldo())
29    }
30 }
```

```
1 class TesteSistemaBancarioComDuplicacao {
2     @Test
3     fun sacarValorDoSaldo() {
4         val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
5         val sistemaBancario = SistemaBancario()
6         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
7         val trindade = bb.criarAgencia("Trindade")
8         val maria = trindade.criarConta("Maria", "000.000.000-00")
9         val cincoReais = casaDaMoeda.emitir(5)
10        maria.depositar(cincoReais)
11        val saque = maria.sacar(cincoReais)
12        assertTrue(saque.sucesso())
13        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
14    }
15
16    @Test
17    fun sacarMaisQueOSaldo() {
18        val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
19        val sistemaBancario = SistemaBancario()
20        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
21        val trindade = bb.criarAgencia("Trindade")
22        val maria = trindade.criarConta("Maria", "000.000.000-00")
23        val cincoReais = casaDaMoeda.emitir(5)
24        maria.depositar(cincoReais)
25        val dezReais = casaDaMoeda.emitir(10)
26        val saque = maria.sacar(dezReais)
27        assertFalse(saque.sucesso())
28        assertEquals(cincoReais, maria.calcularSaldo())
29    }
30 }
```

A aplicação manual de técnicas de reuso de código é suscetível a erros de refatoração e demanda um esforço de desenvolvimento que, por vezes, supera o ganho obtido com a melhoria na manutenibilidade (Landhäußer e Tichy, 2012; Meszaros 2007).

Hipótese de pesquisa (o quê): é possível reduzir a duplicação de código de teste e, ao mesmo tempo, reduzir o esforço total de desenvolvimento dos testes sem que o processo esteja suscetível a erros ou inconsistências.

Hipótese de pesquisa (como): através de um método de refatoração automática de classes teste que utiliza métricas de similaridade para identificação de testes com código em comum e algoritmos de agrupamento para a criação de grupos de testes similares entre si.

Fundamentação Teórica

Para que uma determinada parte do sistema possa ser verificada, é necessário que o sistema em teste (*System Under Test* – SUT) esteja no estado desejado.



```
1 class TesteCriarBancoComConfiguracaoLocal {  
2     @Test  
3     fun criarBanco() {  
4         val sistemaBancario = SistemaBancario()  
5         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)  
6         assertEquals("Banco do Brasil", bb.obterNome())  
7         assertEquals(Moeda.BRL, bb.obterMoeda())  
8     }  
9 }
```



```
1 class TesteCriarBancoComConfiguracaoImplicita {  
2     private lateinit var bb: Banco  
3  
4     @BeforeEach  
5     fun configurar() {  
6         val sistemaBancario = SistemaBancario()  
7         bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)  
8     }  
9  
10    @Test  
11    fun criarBanco() {  
12        assertEquals("Banco do Brasil", bb.obterNome())  
13        assertEquals(Moeda.BRL, bb.obterMoeda())  
14    }  
15 }
```




```
1 class TesteCriarBancoComConfiguracaoDelegada {
2     @Test
3     fun criarBanco() {
4         val bb = Auxiliar.criarBancoDoBrasil()
5         assertEquals("Banco do Brasil", bb.obterNome())
6         assertEquals(Moeda.BRL, bb.obterMoeda())
7     }
8
9     /** Em Kotlin um singleton pode ser construído por uma
10     sintaxe especial usando palavra reservada object. */
11     object Auxiliar {
12         fun criarBancoDoBrasil(): Banco {
13             val sistemaBancario = SistemaBancario()
14             return sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
15         }
16     }
17 }
```

Métricas de similaridade são funções que quantificam o grau de semelhança ou dissimilaridade entre objetos em um conjunto de dados.

- Métricas baseadas em medidas do código.
- Métricas baseadas em texto.
- Métricas baseadas em tokens.
- Métricas baseadas em árvores.
- Métricas baseadas em grafo.

Algoritmos de agrupamento agrupam dados com base em métricas de similaridade.

- Métodos hierárquicos.
- Métodos de particionamento.
- Métodos baseados em densidade.
- Métodos baseados em modelo.
- Métodos baseados em grade.
- Métodos de *soft computing*.

Estado da Arte

Para levantamento do estado da arte foi realizada uma revisão exploratória e um estudo sistemático.

Introdução



Problema de Pesquisa



Fundamentação Teórica



Estado da Arte



Proposta



Conclusão



A revisão exploratória da literatura teve como objetivo aprofundar o conhecimento sobre reuso de código de teste e métricas de similaridade, bem como identificar intersecções entre essas áreas.

Trabalho	Abordagem/Ferramenta
Meszaros (2007)	Configuração implícita
Meszaros (2007)	Configuração delegada
Christensen et al. (2006)	DbUnit
Longo et al. (2015)	Picon
Silva e Vilain (2017)	Estória

Introdução

Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão

Aplicação de métricas de similaridade no contexto de teste (1): redução de suíte de teste tem como objetivo acelerar a execução de testes de regressão através da seleção de apenas um subconjunto dos testes de uma dada suíte (Harrold, Gupta e Soffa, 1993).

Aplicação de métricas de similaridade no contexto de teste (2): priorização de caso de teste tem como objetivo ordenar o conjunto de casos de teste de acordo com algum critério de ordenação estabelecido (Wong et al., 1997).

Trabalho	Objetivo
Ledru et al. (2012)	Priorização de casos de teste
Miranda et al. (2018)	Priorização de casos de teste
Hemmati, Arcuri e Briandi (2010)	Redução de suíte de teste

Introdução



Problema de Pesquisa



Fundamentação Teórica



Estado da Arte



Proposta



Conclusão



O objetivo do estudo sistemático foi identificar trabalhos com o mesmo objetivo principal que o nosso, isto é, a refatoração automática de casos de teste, bem como identificar a originalidade de nossa proposta.

Introdução



Problema de Pesquisa



Fundamentação Teórica



Estado da Arte

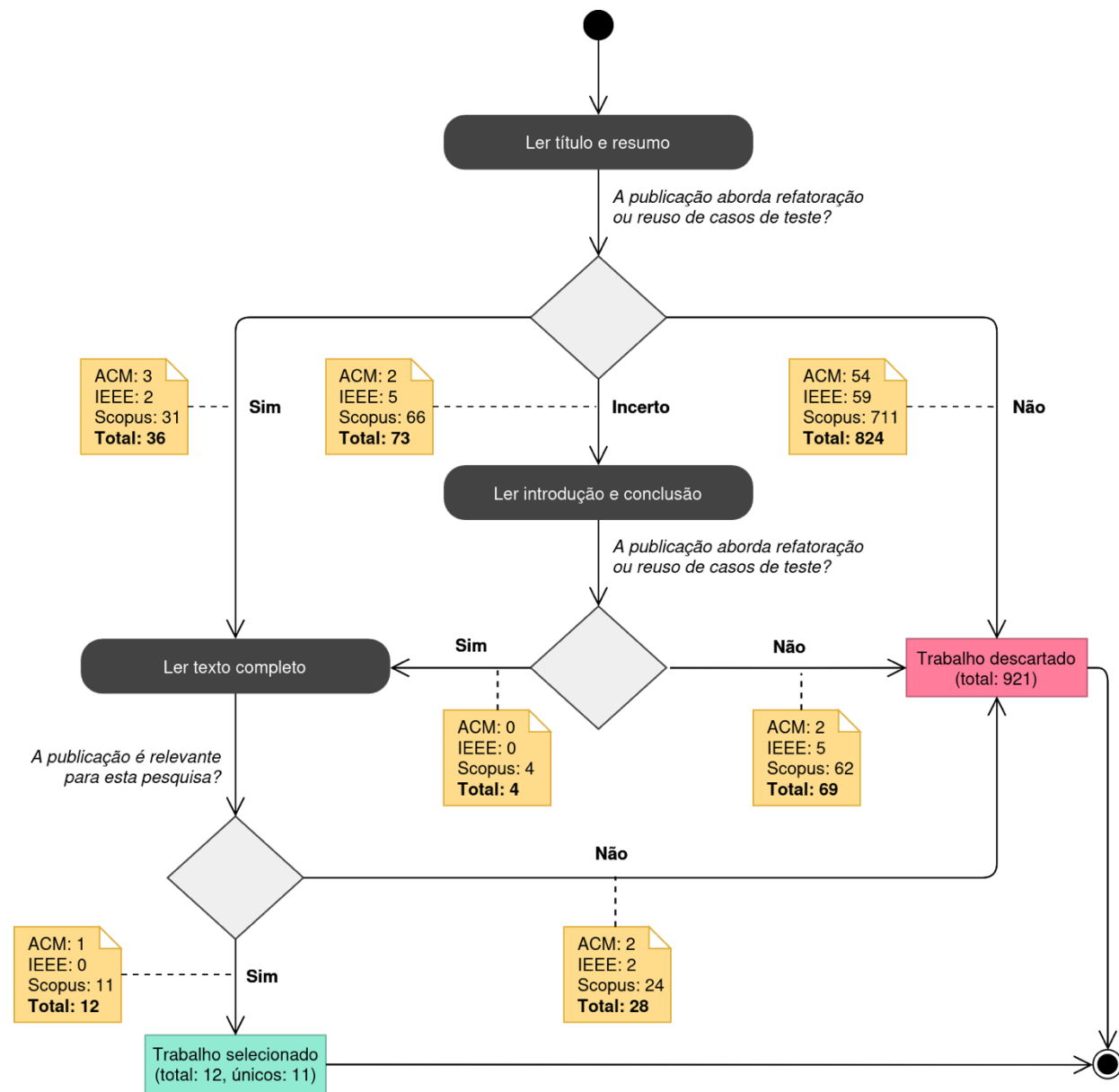


Proposta



Conclusão





Introdução

Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão

Trabalho	Contexto
Shi, Huang e Wan (2022)	Reuso orientado a agrupamento para casos de teste escritos em linguagem natural
Hauptmann et al. (2015)	Refatoração de casos de teste escritos em linguagem natural (extração de passos similares)
Bernard et al. (2020)	Refatoração de casos de teste escritos em linguagem natural (extração de passos similares e parametrização)
Wang et al. (2021, 2022a, 2022b)	Substituição de herança como mecanismo para falsear unidades em teste por bibliotecas de <i>mock</i>
Santana et al. (2022)	Refatoração de fedores de teste (<i>assertion roulette</i> e <i>duplicate assert</i>)
Lambiase et al. (2020)	Refatoração de fedores de teste (<i>general fixture</i> , <i>eager test</i> e <i>lack of cohesion of test methods</i>)
Pizzini, Reinhehr and Mulucelli (2022, 2023)	Refatoração de fedores de teste (<i>eager test</i>)
Xuan et al. (2016)	Refatoração do código de teste para análise dinâmica

Introdução

Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão

Proposta

Utilizar métricas de similaridade e algoritmos de agrupamento para remover de forma automática a duplicação de código de teste.

```
1 class TesteSistemaBancarioComDuplicacao {
2     @Test
3     fun sacarValorDoSaldo() {
4         val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
5         val sistemaBancario = SistemaBancario()
6         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
7         val trindade = bb.criarAgencia("Trindade")
8         val maria = trindade.criarConta("Maria", "000.000.000-00")
9         val cincoReais = casaDaMoeda.emitir(5)
10        maria.depositar(cincoReais)
11        val saque = maria.sacar(cincoReais)
12        assertTrue(saque.sucesso())
13        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
14    }
15
16    @Test
17    fun sacarMaisQueOSaldo() {
18        val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
19        val sistemaBancario = SistemaBancario()
20        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
21        val trindade = bb.criarAgencia("Trindade")
22        val maria = trindade.criarConta("Maria", "000.000.000-00")
23        val cincoReais = casaDaMoeda.emitir(5)
24        maria.depositar(cincoReais)
25        val dezReais = casaDaMoeda.emitir(10)
26        val saque = maria.sacar(dezReais)
27        assertFalse(saque.sucesso())
28        assertEquals(cincoReais, maria.calcularSaldo())
29    }
30 }
```



```

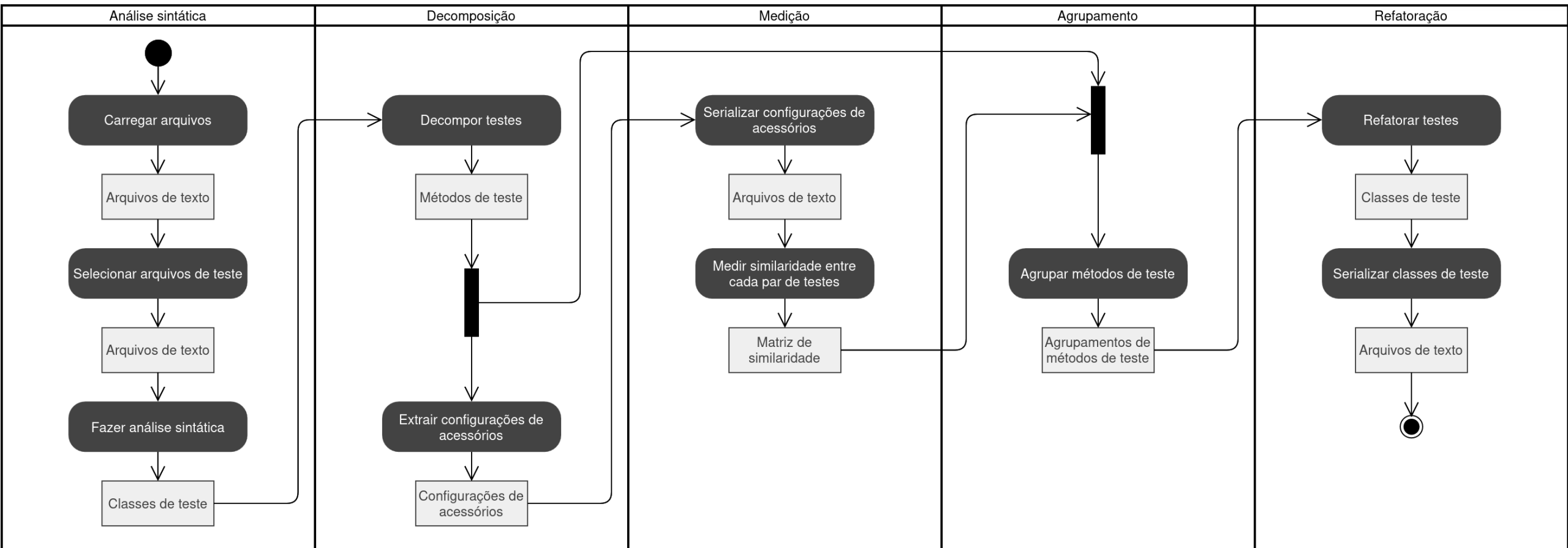
1 class TesteSistemaBancarioSemDuplicacao {
2     private lateinit var casaDaMoeda: CasaDaMoeda
3     private lateinit var cincoReais: Dinheiro
4     private lateinit var maria: Conta
5
6     @BeforeEach
7     fun configurar() {
8         casaDaMoeda = CasaDaMoeda(Moeda.BRL)
9         val sistemaBancario = SistemaBancario()
10        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
11        val trindade = bb.criarAgencia("Trindade")
12        maria = trindade.criarConta("Maria", "000.000.000-00")
13        cincoReais = casaDaMoeda.emitir(5)
14        maria.depositar(cincoReais)
15    }
16
17    @Test
18    fun sacarValorDoSaldo() {
19        val saque = maria.sacar(cincoReais)
20        assertTrue(saque.sucesso())
21        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
22    }
23
24    @Test
25    fun sacarMaisQueOSaldo() {
26        val dezReais = casaDaMoeda.emitir(10)
27        val saque = maria.sacar(dezReais)
28        assertFalse(saque.sucesso())
29        assertEquals(cincoReais, maria.calcularSaldo())
30    }
31 }

```



```
1 class TesteSacarDeChequeEspecial {
2     @Test
3     fun sacarDeChequeEspecial() {
4         val casaDaMoeda = CasaDaMoeda(Moeda.BRL)
5         val sistemaBancario = SistemaBancario()
6         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
7         val trindade = bb.criarAgencia("Trindade")
8         val maria = trindade.criarConta("Maria", "000.000.000-00")
9         val dezReais = casaDaMoeda.emitir(10)
10        maria.fixarLimiteDeChequeEspecial(dezReais)
11        val cincoReais = casaDaMoeda.emitir(5)
12        val saque = maria.sacar(cincoReais)
13        assertTrue(saque.sucesso())
14        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
15        assertEquals(cincoReais, maria.calcularSaldoDeChequeEspecial())
16    }
17 }
```

```
1 class TesteSistemaBancarioSemDuplicacao {
2     private lateinit var casaDaMoeda: CasaDaMoeda
3     private lateinit var cincoReais: Dinheiro
4     private lateinit var maria: Conta
5
6     @BeforeEach
7     fun configurar() {
8         casaDaMoeda = CasaDaMoeda(Moeda.BRL)
9         val sistemaBancario = SistemaBancario()
10        val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
11        val trindade = bb.criarAgencia("Trindade")
12        maria = trindade.criarConta("Maria", "000.000.000-00")
13        cincoReais = casaDaMoeda.emitir(5)
14        maria.depositar(cincoReais)
15    }
16
17    @Test
18    fun sacarValorDoSaldo() {
19        val saque = maria.sacar(cincoReais)
20        assertTrue(saque.sucesso())
21        assertEquals(casaDaMoeda.zero(), maria.calcularSaldo())
22    }
23
24    @Test
25    fun sacarMaisQueOSaldo() {
26        val dezReais = casaDaMoeda.emitir(10)
27        val saque = maria.sacar(dezReais)
28        assertFalse(saque.sucesso())
29        assertEquals(cincoReais, maria.calcularSaldo())
30    }
31 }
```



Introdução

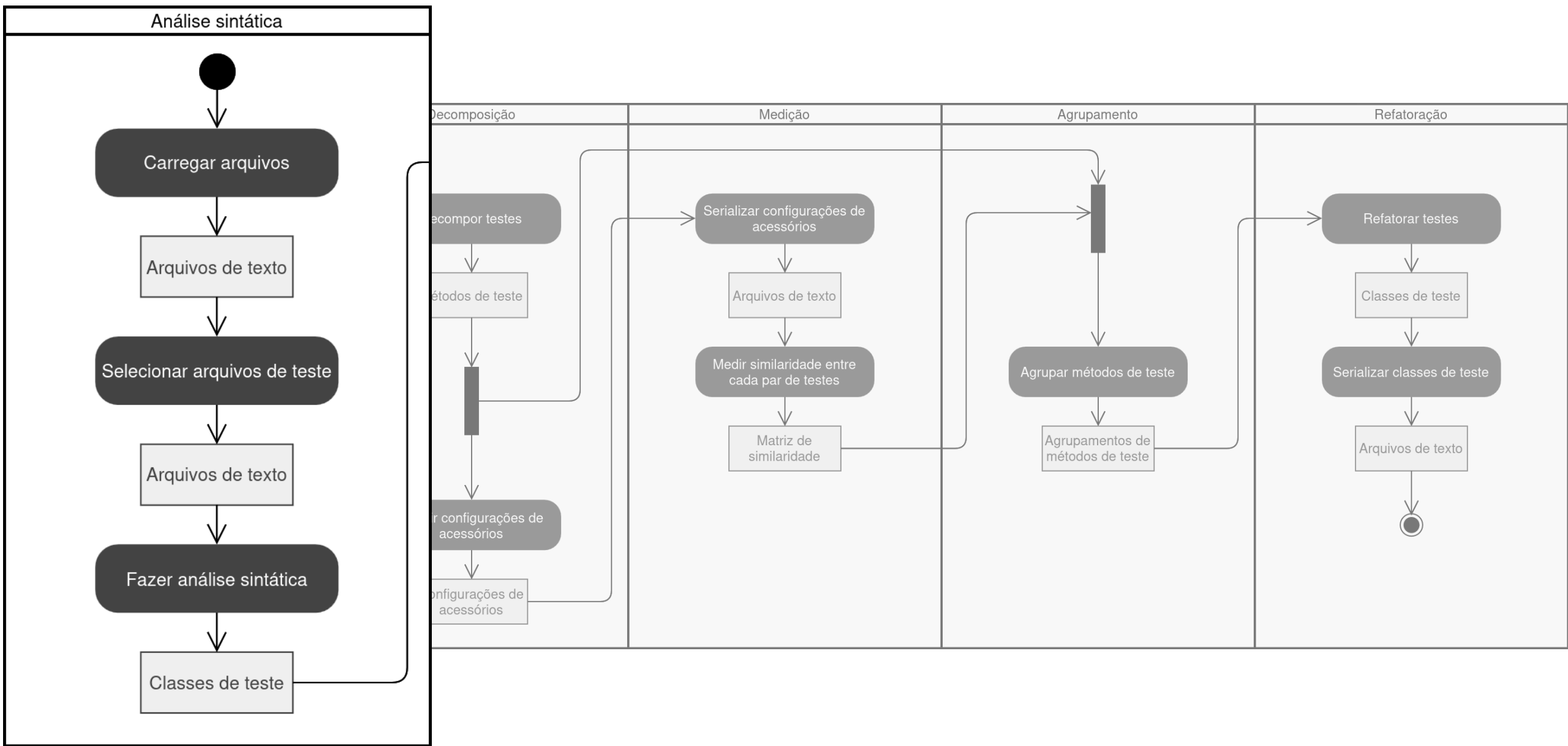
Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão



Introdução

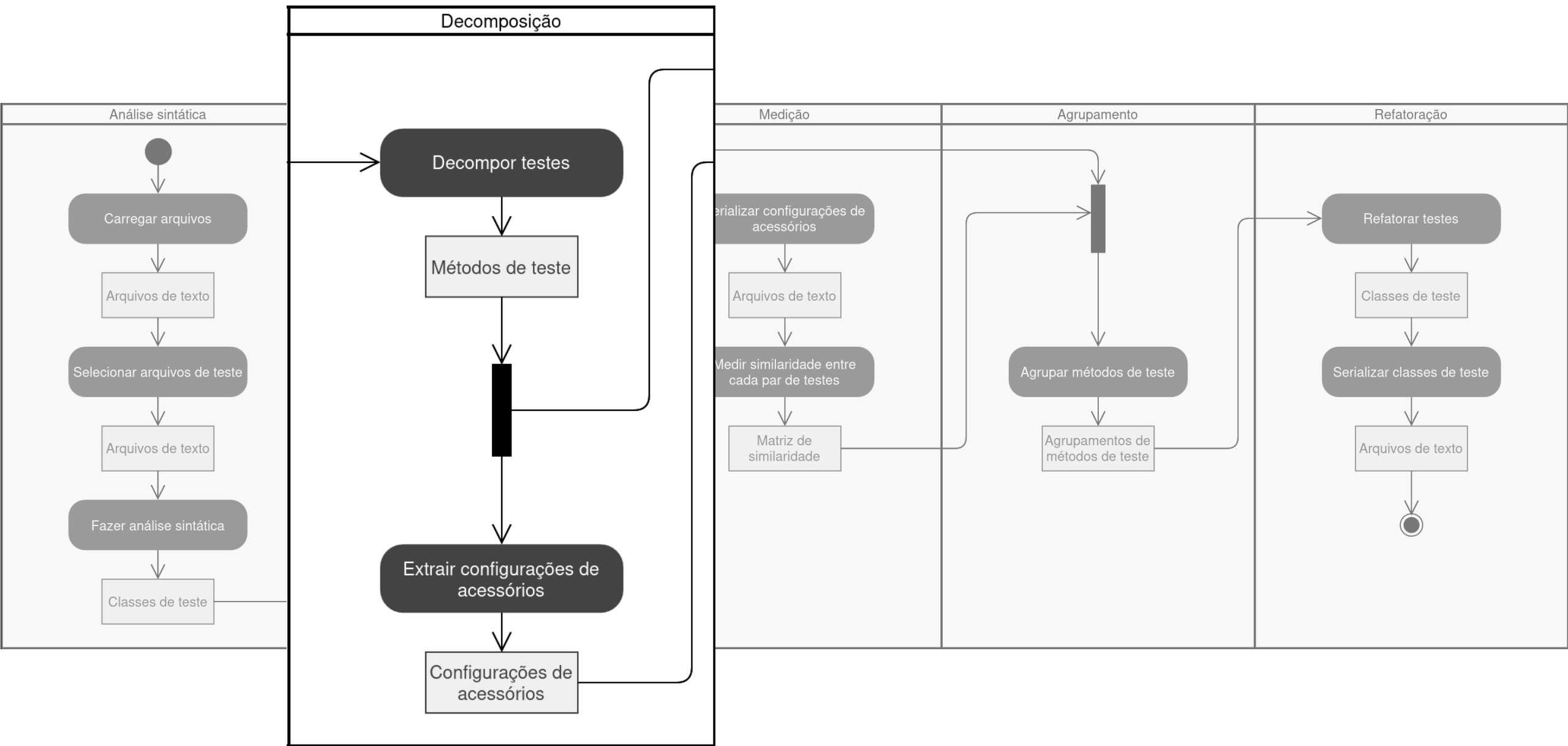
Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão



Introdução

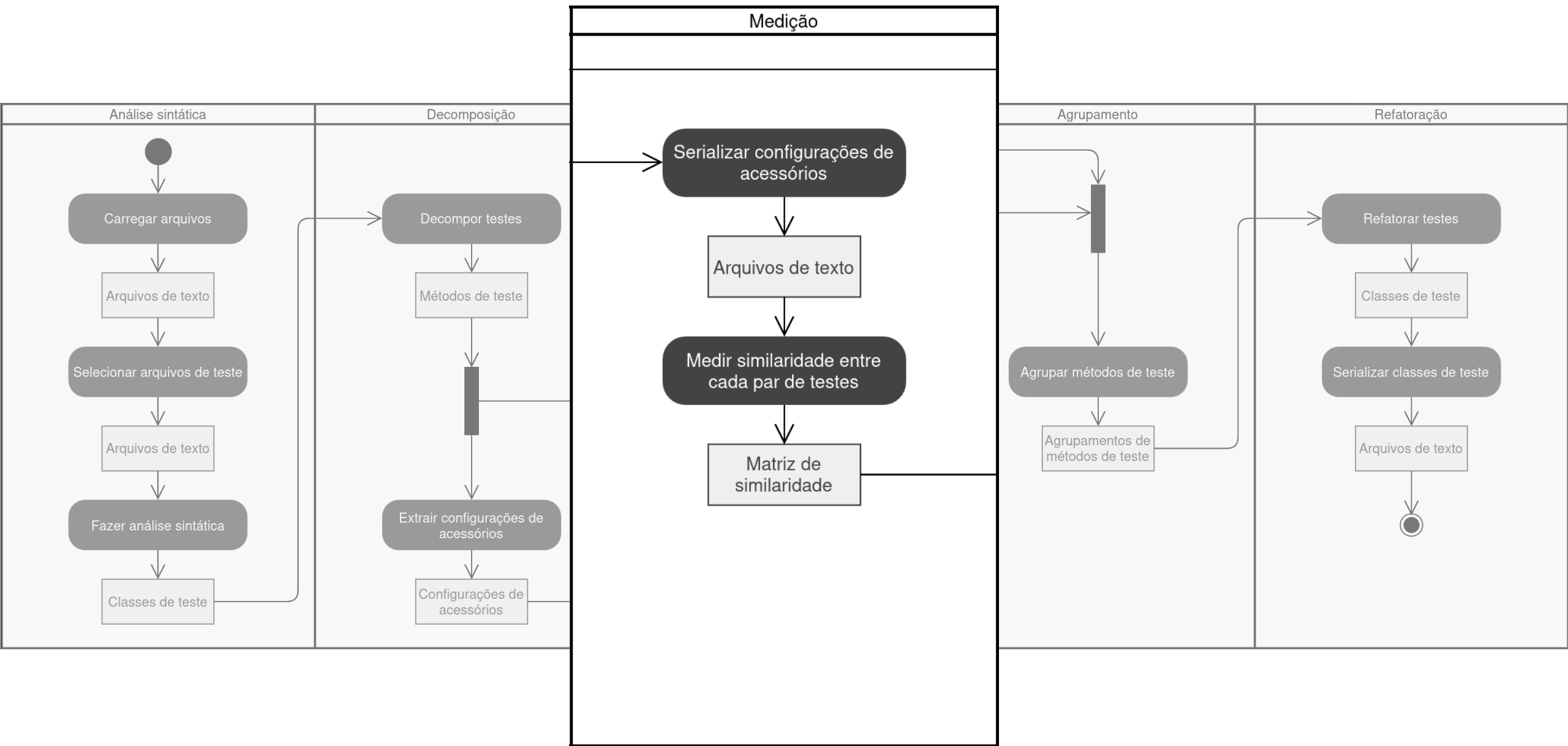
Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão



Introdução

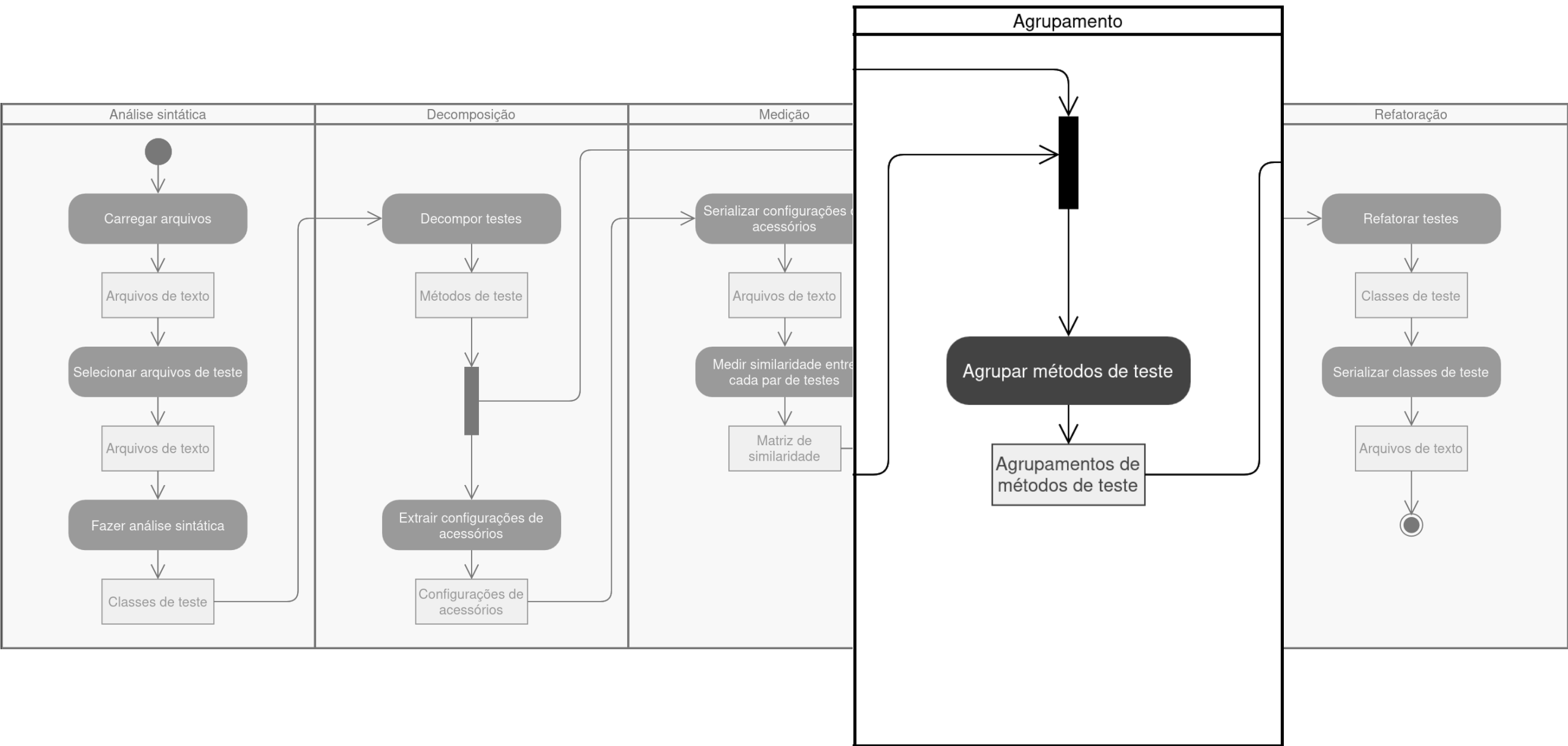
Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão



Introdução

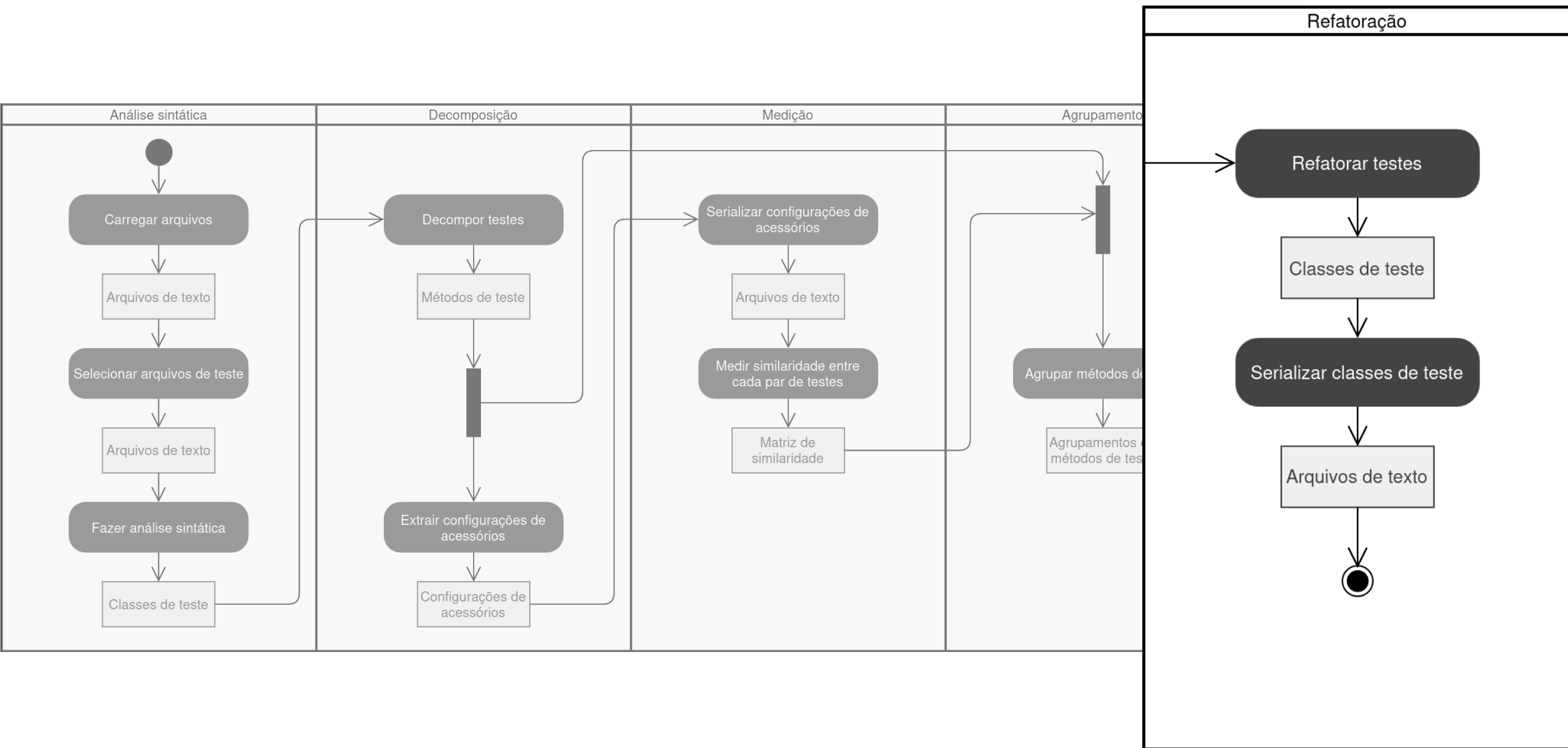
Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão



Introdução

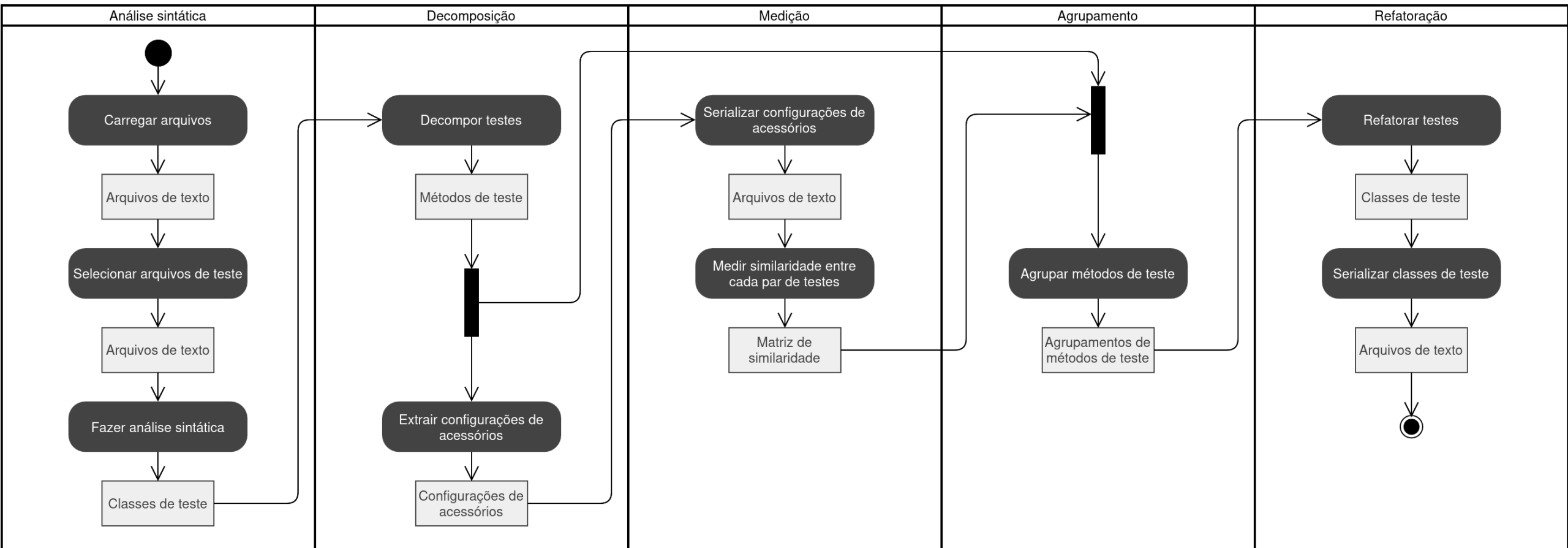
Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão



Introdução

Problema de Pesquisa

Fundamentação Teórica

Estado da Arte

Proposta

Conclusão

Conclusão

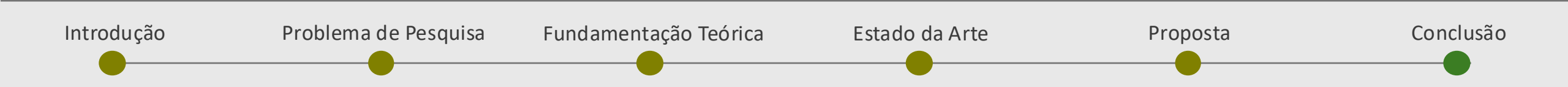
A duplicação do código de teste afeta a manutenibilidade do código de teste e por consequência impacta no esforço total do projeto.

Melhorar a manutenibilidade do código de teste de forma manual pode demandar alto esforço e uma abordagem automática pode diminuir esse problema.

Apesar de existirem abordagens automáticas para melhorar a manutenibilidade de casos de teste, elas ou possuem um escopo limitado ou são aplicadas para casos de teste escritos em linguagem natural.

Apresentamos uma proposta que permite refatorar o código de teste de forma automática.

Atividade	Duração Estimada	Previsão de Início	Previsão de Término
Construção de um <i>framework</i> que implemente a proposta	12 semanas	01/06/2025	31/08/2025
Inclusão de diferentes métricas de similaridade de código no <i>framework</i> proposto	6 semanas	01/07/2025	15/08/2025
Desenvolvimento de uma métrica de similaridade de propósito específico visando identificar pares de teste que podem ser refatorados através da estratégia de configuração implícita	6 semanas	15/07/2025	31/08/2025
Avaliação da métrica de similaridade proposta em comparação com as demais já existentes	6 semanas	15/08/2025	30/09/2025
Elaboração de um artigo apresentando nossa proposta e descobertas	4 semanas	15/09/2025	15/10/2025
Texto e apresentação do Exame de Qualificação de Doutorado	6 semanas	15/10/2025	01/12/2025



FIM

**Obrigado pela presença e por ter dedicado
seu tempo!**